HIGH-SCALE DISTRIBUTED SYSTEMS CASE STUDY

ARCHITECTURE EVOLUTION

Cache Invalidation at Atlassian: From SNS+SQS Push to S3 Pull

Architectural decomposition based on the engineering lecture by **Arpit Bhayani** (*Asli Engineering*, Video ID: `fJNyTNP71MM`) analyzing Atlassian's critical **Tenant Context Service (TCS)**.

> **The Operational Challenge: Invalidation at Six-Nines Scale**
> Atlassian's Tenant Context Service (TCS) provides tenant metadata, permissions, and cryptographic key descriptors for every incoming HTTP request across Jira, Confluence, and Bitbucket. Serving over **600,000 requests per second across 10,000+ compute nodes** with sub-millisecond latency (averaging 11 microseconds at p50), TCS relies on process-local L1 in-memory caches. Because centralized caches (like Redis) introduce network hops, TCS requires near-instantaneous cache invalidation broadcast when tenant configurations mutate.

1. The Context: Why Centralized L2 Caches (Redis/Memcached) Were Rejected

In web-scale distributed microservices, engineers commonly introduce a shared Redis or Memcached cluster as a second-level (L2) cache. However, at Atlassian's operational constraints, shared network caching became untenable:

The Centralized L2 Cache Flaws • Network & Blast Radius

- **Network Hops:** Even inside the same AWS Availability Zone, network round-trips take 1–3 ms, violating the 11 µs target.
- **Single Point of Failure:** If the central cache partition saturates or fails, all upstream services collapse.
- **Connection Churn:** 10,000+ client nodes opening connection pools saturates Redis socket descriptors.

The Node-Local L1 Sidecar Choice • Sub-Millisecond Speed

- **Memory Speed:** Local RAM lookups complete in **hundreds of nanoseconds** via Caffeine in-memory caches.
- **Zero Network I/O on Hit:** Achieves a >99.5% cache hit ratio without touching external dependencies.
- **The Core Trade-off:** Every node maintains its own independent cache; updates require an active **cache invalidation broadcast mechanism**.

2. Invalidation Architectural Models Comparison

Architecture	Data Propagation Model	Message Amplification	Scaling Bottleneck
Short TTL Polling	Local cache expires after fixed duration (e.g., 5 seconds); refetches DynamoDB.	Zero amplification; steady backend polling queries.	High backend read load; serves stale data throughout the TTL window.
SNS + SQS Push Fanout	TCS publishes mutate events to SNS; SNS fans out to an individual SQS queue per compute node.	**O(N) Amplification:** 1 write event triggers 10,000+ SQS message creations.	Massive AWS cost, API rate limits, queue sprawl, and queue lag under traffic surges.
S3-Based Pull Batching	TCS batches mutations and writes versioned changelog objects to S3. Sidecars periodically poll S3.	Zero queue fanout; bounded HTTP GET requests hitting S3 prefix caches.	Slightly higher invalidation latency (1–2 seconds) balanced by extreme stability and minimal cost.

3. The Initial Design: The SNS + SQS Push Fanout Architecture

To achieve real-time cache invalidation, Atlassian initially implemented an AWS-native publish-subscribe fanout pipeline. Whenever a tenant configuration changed, the parent TCS published an invalidation message to an SNS topic:

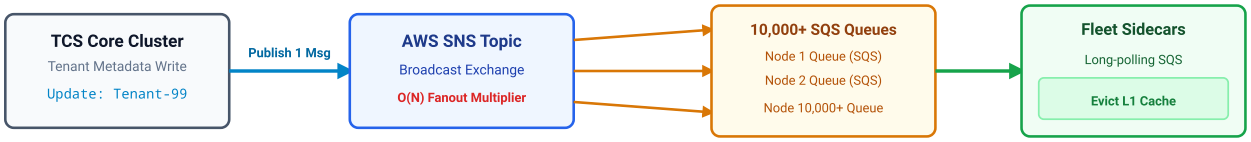


Figure 1: The SNS + SQS fanout pipeline showing how one metadata mutation is amplified across thousands of node queues.

The Mechanics of the Push Model

Each application instance booted with a local sidecar that dynamically provisioned its own dedicated SQS queue and subscribed it to the regional SNS topic. When metadata changed, SNS pushed the event to all active SQS queues, prompting sidecars to evict stale keys locally.

The Amplification Factor Breakdown

If 100 metadata updates occur per second across Atlassian's estate and there are 10,800 active compute nodes, the system must generate:
100 × 10,800 = 1,080,000 SQS messages / sec. This turns a minor database update rate into a monumental messaging deluge.

4. The Breaking Point: Why SNS + SQS Failed at Scale

As Atlassian continued scaling its cloud fleet, the operational flaws of the push topology surfaced:

The Three Crippling Failure Modes of SQS Fanout

1. Explosive Infrastructure Costs: AWS charges for every SQS API call (Send, Receive, Delete). At over 1 million message operations per second, monthly AWS messaging bills climbed uncontrollably into hundreds of thousands of dollars.
2. Node Lifecycle & Queue Sprawl: In auto-scaling Kubernetes clusters, ephemeral pods spin up and down constantly. Lingering orphaned queues piled up, hitting AWS account-level SQS limits and causing management thrash.
3. Message Backlogs and Slow Consumers: If a subset of sidecars encountered CPU throttling or GC pauses, their individual SQS queues backed up, delaying cache invalidation and increasing the risk of split-brain stale reads.

5. The Migration: The Bespoke S3 Pull/Polling Architecture

To eliminate the O(N) message amplification crisis, Atlassian replaced the entire SNS+SQS push topology with an elegant, highly resilient **S3-based pull mechanism**:



Figure 2: The decoupled S3 pull pattern: batching invalidations into immutable objects eliminates individual message fanout.

Batching & Micro-Changelogs

Instead of dispatching discrete messages per key mutation, TCS coalesces all invalidation events occurring over a short sliding window into a single compressed changelog file written to S3, versioned by monotonic timestamp epochs.

S3 HTTP Caching & Cost Collapse

S3 natively scales to thousands of concurrent GET operations per prefix. Sidecars simply check if a newer epoch exists using conditional `If-Modified-Since` headers. Monthly cloud messaging costs were reduced by over 90%.

6. Architectural Lessons & Key Engineering Takeaways

> **The First-Principles Systems Takeaway**
>
> 1. **Push vs. Pull Trade-off:** Push architectures (SNS+SQS) minimize propagation latency to milliseconds but suffer fatal O(N) cost and queue management overhead at high consumer counts.
> 2. **Pragmatic Latency Budgets:** Tenant metadata mutations tolerate a 1–2 second propagation lag. Accepting this tiny lag allowed Atlassian to swap complex push message queues for dead-simple, highly resilient object storage polling.
> 3. **Resilience Through Decoupling:** Object storage has no queues to jam or drop messages during node crashes. If a sidecar restarts, it simply reads the latest S3 epoch and immediately synchronizes its cache state.